

워드 임베딩

자연어 처리 (COSE461)

워드 임베딩

단어 임베딩은 단어와 그 의미 사이의 관계를 벡터 공간에 매핑하는 방법입니다. 이는 분포 가설에 기반하여, 단어의 의미는 그 주변 단어들의 분포에 의해 결정된다는 가정을 따릅니다. 단어 임베딩은 희소 벡터 대 밀집 벡터, 카운트 벡터, 원시 빈도수, 가중 함수, 점별 상호 정보량(PMI), 양의 점별 상호 정보량(PPMI), $\max(\log_2 0)$, $P(w, \text{context})$, $P(w)$, $P(w, c)$, $P(x)P(y)$, 코사인 유사도, 코사인 각도, 내적, $[-1, 1]$, $[-\infty, +\infty]$, $[-$

Buru Chang, Ph.D.

Assistant Professor

컴퓨터 과학 및 공학 학과

한국대학교, 서울, 대한민국



워드 임베딩

클래스 목표

단어 표현 이해

- 워드 임베딩 = 단어 의미의 벡터 표현

추천 독서 교과서 6.2-6.4, 6.6

워드투벡은 단어를 짧은(50-300 차원) & 밀집(실수값) 벡터로 표현합니다.

추천 도서

6.8, 6.10-6.12

단어 벡터 공간에서의 단어 표현의 효율적 추정 (원본 워드투벡 논문)

단어와 구의 분산 표현 및

그 구성성 (부정 샘플링)

Natural Language Processing (COSE461)

워드 임베딩

소개

의미에 대한 큰 아이디어: 유사성에 초점을 맞춘 모델

0.290

-0.441

0.762

0.982

각 단어는 벡터와 동일합니다.

-0.124

0.430

v dog

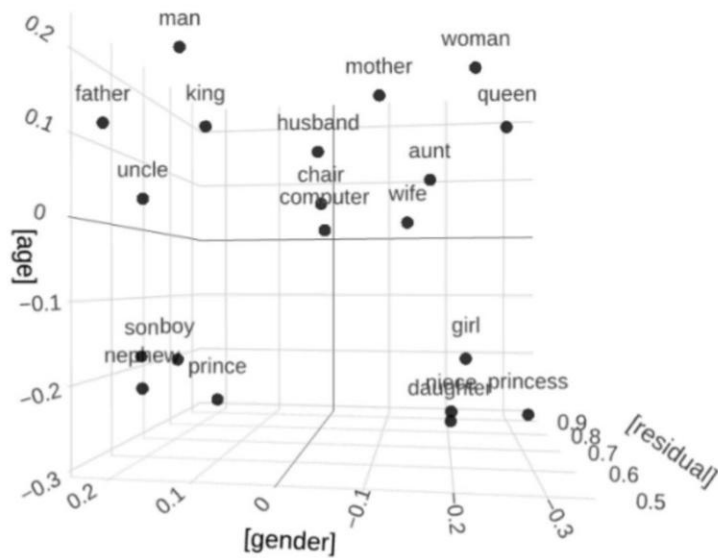
-0.200

0.329

Vlanguage

$$v_{\text{cat}} = \begin{pmatrix} -0.224 \\ 0.130 \\ -0.290 \\ 0.276 \end{pmatrix}$$

유사한 단어는 벡터 공간에서 "근처에 있다"입니다.



Natural Language Processing (COSE461)

워드 임베딩

소개

NLP 모델에서 단어를 어떻게 처리합니까?

n-그램 모델

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i | w_{i-1})$$

$$P(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i) + \alpha}{C(w_{i-1}) + \alpha |\mathcal{V}|}$$

각 단어는 어휘의 문자열 또는 인덱스 w_i 일 뿐이다

cat = $\mathcal{V}$ 에서 5 번째 단어

dog = $\mathcal{V}$ 에서 10 번째 단어

cats = $\mathcal{V}$ 에서 118 번째 단어

- 나이브 베이즈

$$\hat{P}(w_i | c_j) = \frac{\text{Count}(w_i, c_j) + \alpha}{\sum_{w \in \mathcal{V}} \text{Count}(w, c_j) + \alpha |\mathcal{V}|}$$

Natural Language Processing (COSE461)

워드 임베딩

소개

How do we handle words in NLP models?

로지스틱 회귀와 규칙 기반 표현

Var	Definition	Value in Fig. 5.2
x_1	count(positive lexicon words $\in$ doc)	3
x_2	count(negative lexicon words $\in$ doc)	2
x_3	$\begin{cases} 1 & \text{if "no" } \in \text{ doc} \\ 0 & \text{otherwise} \end{cases}$	1
string match x_4	count(1st and 2nd pronouns $\in$ doc)	3
x_5	$\begin{cases} 1 & \text{if "!" } \in \text{ doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	$\ln(\text{word count of doc})$	$\ln(66) = 4.19$

Natural Language Processing (COSE461)

워드 임베딩

소개

단어는 무엇을 의미하는가?

- 동의어: 소파/소파, 자동차/자동차, 필버트/헤이즐넛

어휘: 어두운/밝은, 상승/하락, 위/아래

- 일부 단어는 동의어가 아니지만 의미의 일부를 공유합니다

cat/dog, car/bicycle, COW /horse

일부 단어는 유사하지 않지만 관련이 있습니다

커피/컵, 집/문, 요리사/메뉴

vanish	disappear	9.8
belief	impression	5.95
muscle	bone	3.65
modest	flexible	0.98
hole	agreement	0.3

SimLex-999

워드 임베딩

소개

자연어 처리 모델에서 단어 의미의 필요성

규칙 기반 희소 표현에서 특징은 단어 정체성(= 문자열)입니다.

특징 5: '이전 단어가 "terrible"이었다'

훈련 세트와 테스트 세트에 정확히 동일한 단어가 포함되어야 합니다.

terrible % horrible

만약 단어 의미를 벡터로 표현할 수 있다면:

이전 단어는 벡터 [35, 22, 17, ...]였습니다.

테스트 세트에서는 비슷한 벡터 [34, 21, 14, ...]를 볼 수 있습니다.

유사한 미확인 단어에 대해 일반화할 수 있습니다.

워드 표현

워드 임베딩

워드 표현

분포 가설

- "단어의 의미는 언어에서의 사용이다" - 비트겐슈타인 PI 43

"만약 A 와 B 가 거의 동일한 환경을 가지고 있다면 우리는 그들을 동의어라고 말한다."

"당신은 그 단어가 함께 유지하는 동료에 의해 그 단어를 알게 될 것이다" - Firth 1957

Natural Language Processing (COSE461)

워드 임베딩

워드 표현

분포 가설

분포 가설: 유사한 맥락에서 발생하는 단어들은 유사한 의미를 갖는 경향이 있다



J.R.Firth 1957

- "단어를 그 단어가 함께 어울리는 회사(동료)로 알 수 있다"
- · 현대 통계 NLP 의 가장 성공적인 아이디어 중 하나!

Joun RUPERT Fintu

단어가 텍스트에 나타날 때, 그 맥락은 근처에 나타나는 단어들의 집합이다

(단어 W 가 텍스트에 나타날 때, 그 맥락은 근처에 나타나는 단어들의 집합이다.)

...정부 부채 문제가 2009 년처럼 은행 위기로 전환되는 것을...

...유럽이 잡동사비를 대체할 통합 은행 규제가 필요하다고 말하는 것을...

...인도가 방금 은행 시스템에 활력을 불어넣은 것을...

이 맥락 단어들은 은행을 나타낼 것입니다

Natural Language Processing (COSE461)

워드 임베딩

워드 표현

분포 가설

분포 가설 : "Ongchoi"

옹채는 마늘과 함께 볶으면 맛있다

옹채는 밥 위에 올리면 훌륭하다

옹채 얹은 짬 소스와 함께 먹는다

•

'Ongchoi'가 무엇을 의미한다고 생각하십니까?

- (a) 짭짤한 간식 (b) 녹색 채소 (c) 알코올 음료 (d) 조리용 소스

Natural Language Processing (COSE461)

워드 임베딩

워드 표현

분포 가설

- Ongchoi

옹채는 시금치, 샐러드 또는 콜라드 그린과 같은 왼쪽 녹색 채소입니다



Natural Language Processing (COSE461)

어떻게 동일한 작업을 계산적으로 수행할 수 있을까요?

- 온그초이의 맥락에서 단어를 세어 보세요
- 해당 맥락에서 다른 단어가 발생하는지 확인하세요

단어의 맥락을 벡터를 사용해 표현할 수 있습니다!

워드 임베딩

워드 표현

단어 및 벡터

각 벡터의 차원은 얼마인가?

첫 번째 해결책

단어의 의미를 표현하기 위해 단어-단어 동시 발생 횟수를 사용하자

각 단어는 해당 행 벡터로 표현된다

단어-단어 동시 발생 카운트 행렬을 사용하여 단어의 의미를 표현합니다. 각 단어는 해당 행 벡터로 표현됩니다.

is traditionally followed by **cherry** pie, a traditional dessert
often mixed, such as **strawberry** rhubarb pie. Apple pie
computer peripherals and personal **digital** assistants. These devices usually
a computer. This includes **information** available on the internet

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

대부분의 항목은 0 입니다 → 희소 벡터

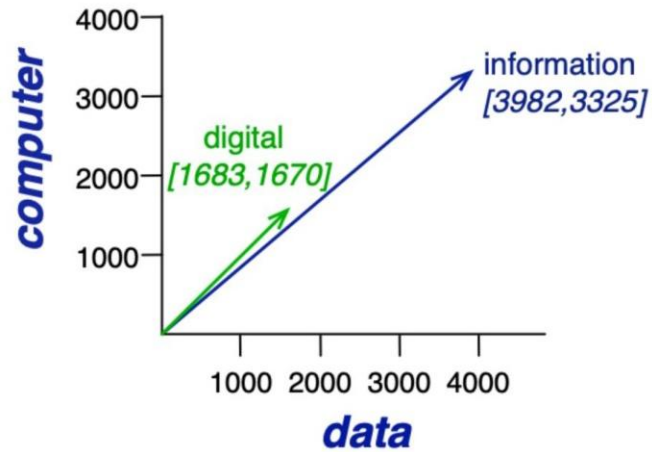
Natural Language Processing (COSE461)

워드 임베딩

워드 표현

유사도 측정

두 벡터 사이의 각도 코사인: 일반적인 유사도 측정 지표 (값이 클수록 두 벡터가 더 유사함)



$u \cdot v$

코사인 유사도

미미

121

$i=1$ $u_i v_i$

코사인 유사도 =

$\frac{u \cdot v}{\|u\| \|v\|}$

코사인 유사도 대신 내적 $u \cdot v$ 를 사용하는 이유는 무엇인가요?

Natural Language Processing (COSE461)

워드 임베딩

워드 표현

유사도 측정

코사인 유사도 범위란 u 와 v 가 카운트 벡터일 때 어떤 범위를 가지는가?

Natural Language Processing (COSE461)

워드 임베딩

워드 표현

이 모델의 문제점은 무엇인가요?

원시 빈도수는 나쁜 표현입니다

"파이"는 "체리" 근처에 많이 나타난다면 분명히 유용하지만, "the", "it", "they"와 같이 지나치게 빈번한 단어도 "체리" 근처에 많이 나타난다. 이러한 단어들은 문맥에 대해 그다지 많은 정보를 제공하지 않는다.

- 해결책: 원시 빈도수 대신 가중 함수를 사용

Natural Language Processing (COSE461)

워드 임베딩

워드 표현

점별 상호 정보량 (PMI)

x 와 y 가 독립적일 때보다 더 많이 또는 더 적게 동시 발생합니까?

$$\text{PMI}(x, y) = \log_2 \frac{P(x, y)}{P(x)P(y)}$$

코사인 유사도는 두 벡터 사이의 각도를 코사인으로 측정한 유사도입니다. 코사인 유사도는 -1 에서 1 사이의 값을 가지며, 1 은 두 벡터가 완전히 같은 방향을 가리킨다는 것을 의미하고, -1 은 완전히 반대 방향을 가리킨다는 것을 의미합니다. 0 은 두 벡터가 직교한다는 것을 의미합니다. 코사인 유사도는 벡터 간의 유사도를 측정하는 데 널리 사용됩니다.

"pie"가 "cherry" 근처에서 우리가 기대하는 것보다 더 자주 발생합니까?

$P(w = \text{cherry}, C = \text{pie})$

$\text{PMI}(w = \text{cherry}, C = \text{pie}) = \log_2$

$P(w = \text{cherry})P(c = \text{pie})$

Natural Language Processing (COSE461)

워드 임베딩

워드 표현

양의 점별 상호 정보량 (PPMI)

- PMI 는 - ∞ 에서 $+\infty$ 까지 범위를 가집니다
- $\text{PMI}(w, c) > 0 \rightarrow P(w, c) > P(w)P(c)$
- $\text{PMI}(w, c) < 0 \rightarrow P(w, c) < P(w)P(c)$
- PMI 의 음수 값은 코퍼스가 매우 크지 않으면 자주 신뢰할 수 없습니다
- "무관계" 점수를 인간의 판단으로 평가할 수 있는지 여부는 불분명합니다

- 간단한 해결책: 모든 음수 PMI 값을 0 으로 대체한다

$$\text{PPMI}(x, y) = \max\left(\log_2 \frac{P(x, y)}{P(x)P(y)}, 0\right)$$

Natural Language Processing (COSE461)

워드 임베딩

워드 표현

PPMI - 실행 예시

$$- P_{ij} = \frac{f_{ij}}{\sum_{i=1}^W \sum_{j=1}^C f_{ij}}, P(w_i) = \frac{\sum_{j=1}^C f_{ij}}{N}, P(c_j) = \frac{\sum_{i=1}^W f_{ij}}{N}$$

3982

- P(w = 정보, c = 데이터) = 0.3399

11716

- P(w = 정보) 0.6575

11716

	computer	data	result	pie	sugar	count(w)
cherry	2	8	9	442	25	486
strawberry	0	0	1	60	19	80
digital	1670	1683	85	5	4	3447
information	3325	3982	378	5	13	7703
count(context)	4997	5673	473	512	61	11716

	p(w,context)					p(w)
	computer	data	result	pie	sugar	p(w)
cherry	0.0002	0.0007	0.0008	0.0377	0.0021	0.0415
strawberry	0.0000	0.0000	0.0001	0.0051	0.0016	0.0068
digital	0.1425	0.1436	0.0073	0.0004	0.0003	0.2942
information	0.2838	0.3399	0.0323	0.0004	0.0011	0.6575
p(context)	0.4265	0.4842	0.0404	0.0437	0.0052	

- 텍스트 코퍼스 1M 토큰을 가지고 있다고 가정하고, 각 단어 W 에 대해 4 개의 단어 앞과 4 개의 단어 뒤를 컨텍스트 c 로 사용한다면, 이러한 확률을 계산하는 분모인 N 은 대략 얼마인가?

(A) 1M (B) 4M (C) 8M (D) 정보가 충분하지 않음

Natural Language Processing (COSE461)

워드 임베딩

워드 표현

PPMI - 실행 예시

- 0.008
- $\text{PMI}(\text{cherry}, \text{result}) = \log_2 = -1.07$
- $0.0415 * 0.0404$
- 0.0073
- 점별 상호 정보량 (디지털, 결과) = $\log_2 = -0.70$
- $0.2942 * 0.0404$
- 결과 PPMI 행렬 (음수는 0 으로 대체)

	p(w,context)					p(w)
	computer	data	result	pie	sugar	p(w)
cherry	0.0002	0.0007	0.0008	0.0377	0.0021	0.0415
strawberry	0.0000	0.0000	0.0001	0.0051	0.0016	0.0068
digital	0.1425	0.1436	0.0073	0.0004	0.0003	0.2942
information	0.2838	0.3399	0.0323	0.0004	0.0011	0.6575
p(context)	0.4265	0.4842	0.0404	0.0437	0.0052	

	computer	data	result	pie	sugar
cherry	0	0	0	4.38	3.30
strawberry	0	0	0	4.10	5.51
digital	0.18	0.01	0	0	0
information	0.02	0.09	0.28	0	0

Natural Language Processing (COSE461)

워드 임베딩

워드 표현

희소 벡터 대 밀집 벡터

단어-단어 발생 행렬에서의 벡터는 다음과 같습니다:

간: 어휘 크기

희소: 대부분이 0 입니다

- 대안: 우리는 단어를 짧은(50-300 차원) & 밀집

(실수값) 벡터로 표현하고 싶다

현대 NLP 시스템의 기반

Natural Language Processing (COSE461)

워드 임베딩

워드 표현

밀집 벡터는 왜 사용하는가?

- 짧은 벡터는 머신 러닝 시스템에서 특성으로 사용하기 더 쉽다

밀집 벡터는 명시적 카운트보다 더 잘 일반화됩니다(실수 공간의 점과 정수 공간의 점)

희소 벡터는 고차 동시 발생을 포착할 수 없습니다.

W1 은 "car"와 함께 발생하고, W2 는 "automobile"과 함께 발생한다.

이들은 유사해야 하지만, "car"와 "automobile"이 서로 다른 차원이기 때문에 그렇지 않다.

실제로 더 잘 작동합니다.

Natural Language Processing (COSE461)

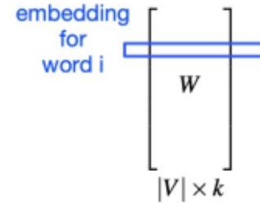
워드 임베딩

워드 표현

어떻게 짧은 밀집 벡터를 얻을 수 있는가?

카운트 기반 방법: 카운트 행렬의 특이값 분해 (특이값 분해)

$$\begin{bmatrix} X \\ |V| \times |V| \end{bmatrix} = \begin{bmatrix} W \\ |V| \times |V| \end{bmatrix} \begin{bmatrix} \sigma_1 & 0 & 0 & \dots & 0 \\ 0 & \sigma_2 & 0 & \dots & 0 \\ 0 & 0 & \sigma_3 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \sigma_V \end{bmatrix} \begin{bmatrix} C \\ |V| \times |V| \end{bmatrix}$$



$$\begin{bmatrix} X \\ |V| \times |V| \end{bmatrix} = \begin{bmatrix} W \\ |V| \times k \end{bmatrix} \begin{bmatrix} \sigma_1 & 0 & 0 & \dots & 0 \\ 0 & \sigma_2 & 0 & \dots & 0 \\ 0 & 0 & \sigma_3 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \sigma_k \end{bmatrix} \begin{bmatrix} C \\ k \times |V| \end{bmatrix}$$

We can approximate the full matrix by only keeping the top k (e.g., 100) singular values!

Natural Language Processing (COSE461)

워드 임베딩

Word Representation

예측 기반 방법:

벡터는 단어 c ("pie")가 단어 W ("cherry")의 맥락에서 나타날 가능성이 있는지를 예측하도록 분류기를 훈련시켜 생성됩니다.

Examples: word2vec (Mikolov et al., 2013), Glove (Pennington et al., 2014), FastText (Bojanowski et al., 2017)

Natural Language Processing (COSE461)

Word2vec and other variants

Word Embedding

워드 임베딩

Learned representations from text for representing words

- Input: a large text corpora, V , d

V : a pre-defined vocabulary

d : dimension of word vectors (e. g. , 300)

Text corpora:

Wikipedia + Gigaword 5: 6B tokens

Twitter: 27B tokens

Common Crawl: 840B tokens

d

- 출력: $f: V \rightarrow \mathbb{R}^d$ -

Each word is represented by a low-dimensional (e.g., $d = 300$), real-values vector

Each coordinate/dimension of the vector doesn't have a particular interpretation

Natural Language Processing (COSE461)

워드 임베딩

Word Embedding

Trained word embeddings available

- 워드투벡: <http://code.google.com/e/p/워드투벡/>
- 글로브: <https://nlp.stanford.edu/projects/글로브/>
- 패스트텍스트: <https://패스트텍스트.cc/>

Download pre-trained word vectors

· Pre-trained word vectors. This data is made available under the Public Domain

Dedication and License v1.0 whose full text can be found at:

<http://www.opendatacomg/licenses/pddl/1.o/>.

- Wikipedia2014 + Gigaword5 (6B tokens, 400K vocab, uncased, 50d, 100d, 200d, & 300d vectors, 822 MB download): glove.6B.zip
- O Common Crawl (42B tokens, 1.9M vocab, uncased, 300d vectors, 1.75 GB download): glove.42B.300d.zip
- O Common Crawl (840B tokens, 2.2M vocab, cased, 300d vectors, 2.03 GB download): glove.840B.300d.zip
- o Twitter (2B tweets, 27B tokens, 1.2M vocab, uncased, 25d, 50d, 100d, & 200d vectors, 1.42 GB download): glove.twitter.27B.zip

Differ in algorithms, text corpora, dimensions, cased/uncased...

Applied to many other languages

Natural Language Processing (COSE461)

Word Embedding

워드 임베딩

- Basic property: similar words have similar vectors

word W^* = "Sweden"

argmax_w cos(e(w), e(w^*))

Word	Cosine Similarity
norway	0.760124
denmark	0.715460
finland	0.620022
switzerland	0.588132
belgium	0.585835

코사인 유사도 범위는 -1 과 1 사이입니다

Natural Language Processing (COSE461)

Word Embedding

Word Embedding

Trained word embeddings available

Nearest words to

frog:

- 1. frogs
- 2. toad
- 3. litoria
- 4. leptodactylidae
- 5. rana
- 6. lizard
- 7. eleutherodactylus



litoria



레프토다크틸리다과



rana



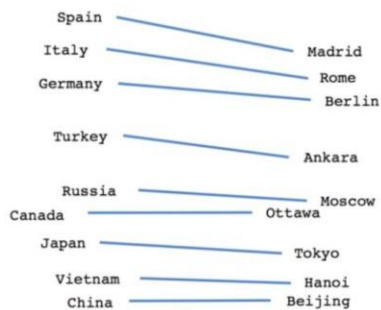
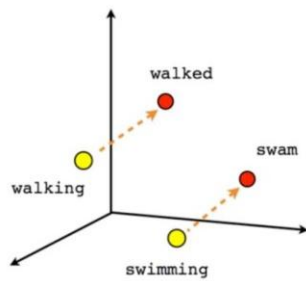
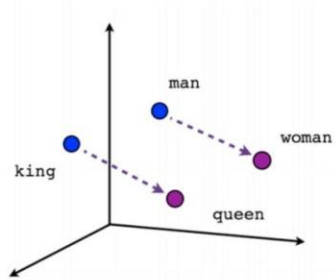
eleutherodactylus

(Pennington et al, 2014): GloVe:Global Vectors for Word Representation Natural Language Processing (COSE461)

Word Embedding

Word Embedding

그들은 몇 가지 다른 좋은 특성도 가지고 있습니다.



Country-Capital

Male-Female

Verb tense

$$v_{\text{man}} - v_{\text{woman}} \approx v_{\text{king}} - v_{\text{queen}}$$

$$v_{\text{Paris}} - v_{\text{France}} \approx v_{\text{Rome}} - v_{\text{Italy}}$$

Word analogy test: $a : a^* :: b : b^*$

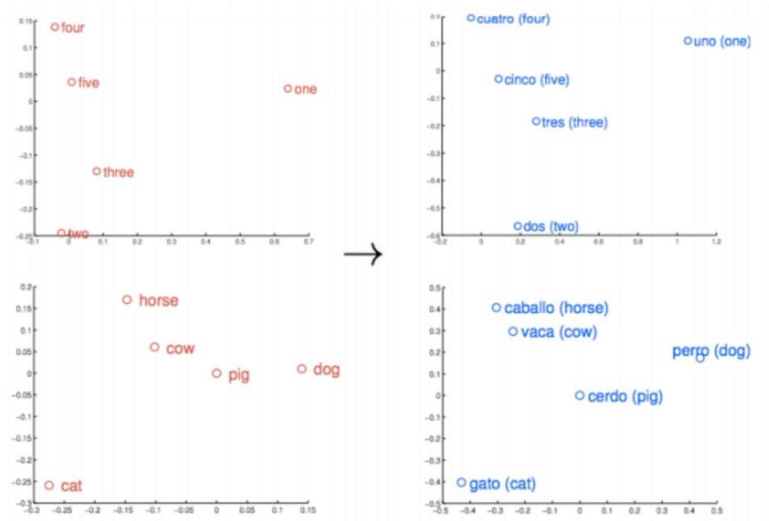
$$b^* = \arg \max_{w \in V} \cos(e(w), e(a^*) - e(a) + e(b))$$

Natural Language Processing (COSE461)

Word Embedding

워드 임베딩

$$v(\text{cuatro}) \approx Wv(\text{four})$$



(Mikolov et al, 2013): Exploiting Similarities among Languages for Machine Translation

Natural Language Processing (COSE461)

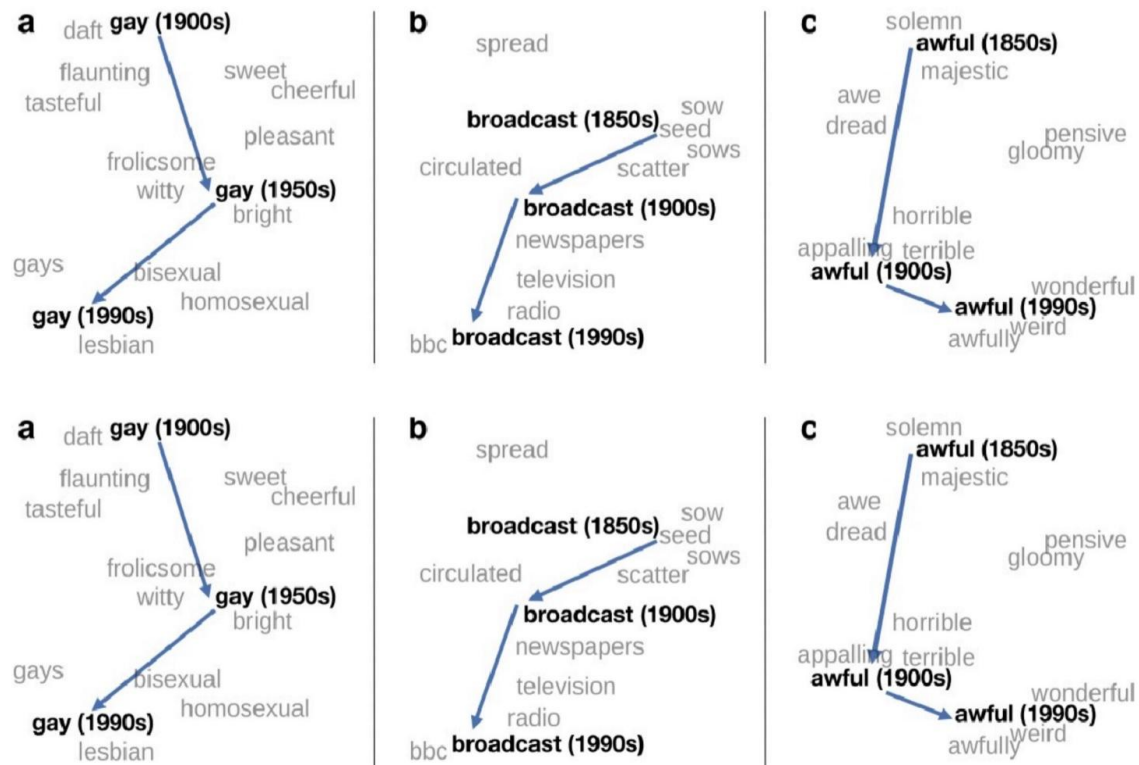
Word Embedding

Word Embedding

Embeddings as a window onto historical semantics

- 역사적 텍스트의 서로 다른 십년 단위로 임베딩을 학습하여 의미가 어떻게 변하는지 확인한다

~30 million books, 1850-1990, Google Books data



William L. Hamilton, Jure Leskovec, and Dan Jurafsky. 2016. Diachronic Word Embeddings Reveal Statistical Laws of Semantic Change. Proceedings of ACL.

Natural Language Processing (COSE461)

Word Embedding

Word Embedding

임베딩은 문화적 편향을 반영합니다

Bolukbasi, Tolga, Kai-Wei Chang, James Y. Zou, Venkatesh Saligrama, and Adam T. Kalai. "Man is to computer

programmer as woman is to homemaker? debiasing word embeddings. " In NeurIPS, pp. 4349-4357. 2016.

Ask "Paris : France :: Tokyo : x

o X = Japan

Ask "father : doctor :: mother : X

o X = 간호사

Ask "man : computer programmer :: woman : x"

o X = homemaker

Algorithms that use embeddings as part of e.g., hiring searches for programmers, might lead to bias in hiring

Natural Language Processing (COSE461)

Word2vec: How does it work?

워드 임베딩

Word2vec

Introduction

- Goal: represent words as short (50-300 dimensional) & dense (real-valued) vectors

Count-based approaches

- · 90 년대부터 사용되었습니다
- · 희소 단어-단어 동시 발생 PPMI
- 행렬
- 특이값 분해로 분해되었습니다
- Prediction-based approaches
- ● Formulated as a machine learning problem
- · Word2vec (Mikolov et al., 2013)
- GloVe (Pennington et al., 2014)

Underlying theory: Distributional Hypothesis (Firth, '57)

"Similar words occur in similar context"

Natural Language Processing (COSE461)

Word Embedding

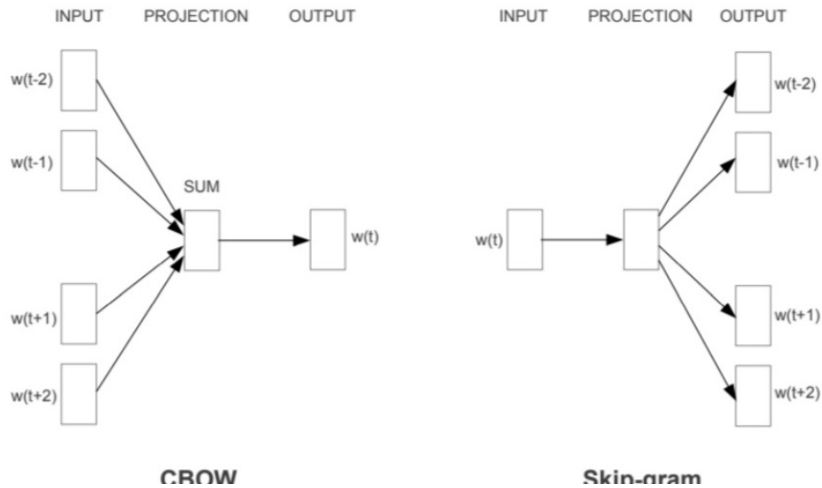
Word2vec

Embeddings as a window onto historical semantics

- (Mikolov et al., 2013a): 벡터 공간에서 단어 표현의 효율적 추정
- (Mikolov et al., 2013b): Distributed Representations of Words and Phrases and their

Compositionality





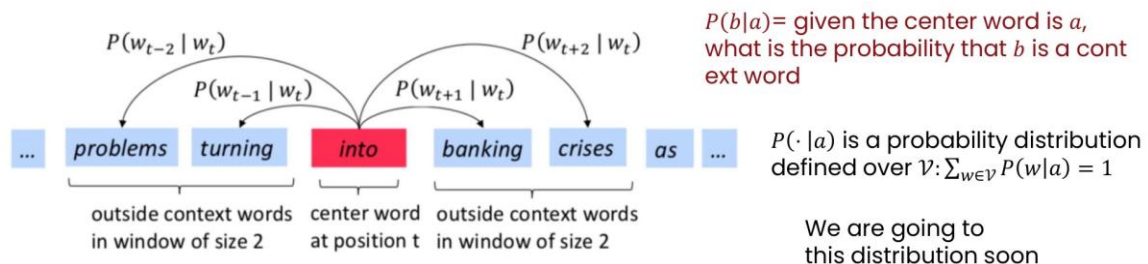
Natural Language Processing (COSE461)

Word Embedding

Word2vec

Skip-gram

- Assume that we have a large corpus $W_1, W_2, \dots, W_T \in V$
- Key idea: use each word to predict other words in its context
- Context: a fixed window of size $2m$ ($m = 2$ in the example) A classification problem

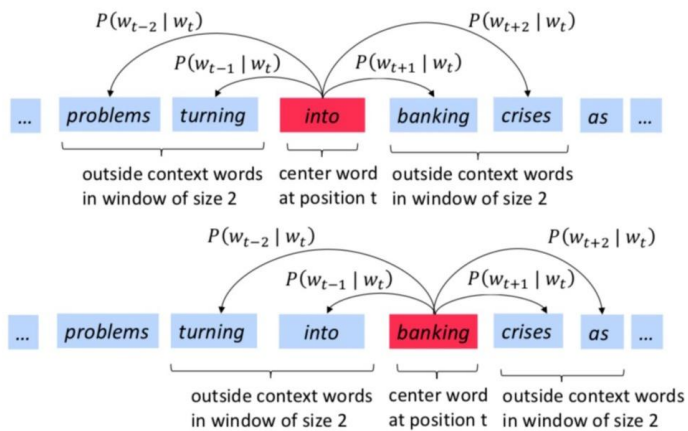


Natural Language Processing (COSE461)

Word Embedding

Word2vec

스킵-그램



Conver into training data:

(into, problems)
(into, turning)
(into, banking)
(into, crises)

(banking, turning)
(banking, into)
(banking, crises)
(banking, as)

- Our goal is to find parameters that can maximize

$$P(\text{problems}|\text{into}) \times P(\text{turning}|\text{into}) \times P(\text{banking}|\text{into}) \times P(\text{crises}|\text{into}) \times$$

$$P(\text{turning}|\text{banking}) \times P(\text{into}|\text{banking}) \times P(\text{crises}|\text{banking}) \times P(\text{on}|\text{banking})$$

Word Embedding

Word2vec

Skip-gram: 목적 함수

- For each position $t = 1, 2, \dots, T$, predict context words within context size m , given center word W_t :

$$L() = \sum_{t=1}^T \sum_{j=-m}^m \sum_{j \neq 0} P(W_{t+j} | W_t; \theta)$$

all the parameters to be optimized

- It is equivalent to minimizing the (average) negative log likelihood:

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log P(w_{t+j} | w_t; \theta)$$

Natural Language Processing (COSE461)

Word Embedding

워드투벡

How to define $P(w_{t+j} | w_t)$?

- Use two sets of vectors for each word in the vocabulary

$\mathbf{u}_a \in \mathbb{R}^d$: vector for center word a , $\mathbf{u}_a \in V$

$\mathbf{u}_a$

$\mathbf{v}_b \in \mathbb{R}^d$: vector for context word b , $\mathbf{v}_b \in V$

- Use inner product $\mathbf{u}_a \cdot \mathbf{v}_b$ to measure how likely word a appears with context word b

$$P(w_{t+j} | w_t) = \frac{\exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_{w_{t+j}})}{\sum_{k \in \mathcal{V}} \exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_k)}$$

소프트맥스는 다항 로지스틱 회귀에서 본 적이 있습니다

$P(\cdot | a)$ 는 V 위에서 정의된 확률 분포입니다

Natural Language Processing (COSE461)

Word Embedding

Word2vec

VS multinomial logistic regression

- Essentially a $|V|$ -way classification problem

$$\exp(w_c \cdot x + b_c)$$

다항 로지스틱 회귀: $P(y = c|x)$

$$E_{m=1} \exp(w_j \cdot x + b_j)$$

it is reduced to a multinomial logistic regression problem.

- If we fix u_{wt}

- However, since we have to learn both u and V together, the training objective is

non-convex.

$$\exp(u_{wt} \cdot V_{wt+j})$$

$$P(w_{t+j}|w_t)$$

$$E_{K \sim V} \exp(u_{wt} \cdot V_k)$$

Natural Language Processing (COSE461)

워드 임베딩

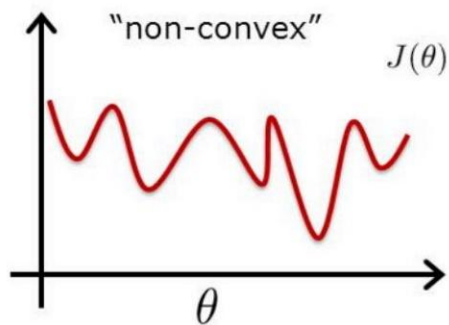
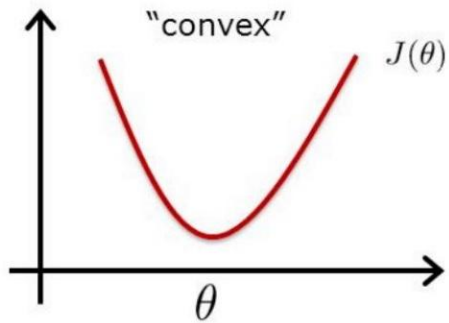
Word2vec

- It is hard to find a global minimum

- But can still use stochastic gradient descent to optimize :

$$t+1 \leftarrow t - \eta \nabla J()$$

-



Natural Language Processing (COSE461)

워드 임베딩

Word2vec

Important note

T

$$1 \exp \mathbf{u}^T \mathbf{w} \cdot \mathbf{V}^T \mathbf{w} + \mathbf{j}$$

$$J() = > 2 \log$$

T

$$\text{EKEV} \exp (\mathbf{u}^T \mathbf{w} \cdot \mathbf{V}^T \mathbf{w} + \mathbf{j}$$

$$t=1 -m \leq j \leq m, j \neq 0$$

- In this formulation, we don't care about the classification task itself like we do for the logistic regression model we saw previously.

- The key point is that the parameters used to optimize this training objective - when the training corpus is large enough - can give use very good representations of words, following the principle of distributional hypothesis.

Natural Language Processing (COSE461)

워드 임베딩

Word2vec

How many parameters in this model?

T

$\frac{1}{2} \sum_{t=1}^T \sum_{j=1}^d \exp(u_{wt} \cdot V_{wt+j})$

$J() = \frac{1}{2} \sum_{t=1}^T \sum_{j=1}^d \log$

T

$EKEV \exp(u_{wt} \cdot V_k)$

$t=1 \quad -m \leq j \leq m, j \neq 0$

- How many parameters does this model have (i.e., what is size of Θ)?

- (A) 디지털 레벨
- (B) 2 차원 미
- (c) 2 차원 벡터
- (D) 2 차원 벡터 차원

d = dimension of each vector

Natural Language Processing (COSE461)

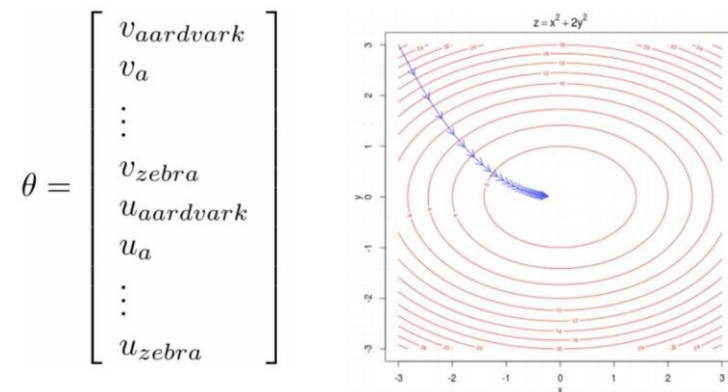
Word Embedding

Word2vec

How to train this model?

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log \frac{\exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_{w_{t+j}})}{\sum_{k \in \mathcal{V}} \exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_k)}$$

- To train such a model, we need to compute the vector gradient $\nabla J()$ =?
- Remember that θ represents all $2d|\mathcal{V}|$ model parameters, in one vector.



Natural Language Processing (COSE461)

워드 임베딩

Word2vec

Vectorized gradients

af

$$f(x) = X \cdot a, X, a \in \mathbb{R}^n, = a$$

ax

$$f = x_1 a_1 + x_2 a_2 + \dots + x_n a_n$$

$$\mathbf{a}_f = \sum_{i=1}^n x_i \mathbf{a}_i$$

$$\frac{\partial f}{\partial x_1} = \mathbf{a}_1, \frac{\partial f}{\partial x_2} = \mathbf{a}_2, \dots, \frac{\partial f}{\partial x_n} = \mathbf{a}_n$$

$\mathbf{W}[\mathbf{a}]$

$\frac{\partial f}{\partial \mathbf{a}_1} = \mathbf{x}_1, \quad \frac{\partial f}{\partial \mathbf{a}_2} = \mathbf{x}_2$

$$\frac{\partial f}{\partial \mathbf{x}} = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]$$

Natural Language Processing (COSE461)

Word Embedding

Word2vec

Vectorized gradients: exercises

Let $f = \exp(\mathbf{w} \cdot \mathbf{x})$, what is the value of $\frac{\partial f}{\partial \mathbf{x}}$? $\mathbf{w}, \mathbf{x} \in \mathbb{R}^n$

ax

- (A) $\mathbf{w}$
- (B) $\exp(\mathbf{w} \cdot \mathbf{x})$
- (C) $\exp(\mathbf{w} \cdot \mathbf{x})\mathbf{w}$
- (D) $\mathbf{x}$

Natural Language Processing (COSE461)

Word Embedding

Word2vec

Let's compute gradients for word2vec

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log \frac{\exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_{w_{t+j}})}{\sum_{k \in \mathcal{V}} \exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_k)}$$

- 중심/문맥 단어 쌍 (t, c) 하나를 고려해 보자:

$$y = -\log \left(\frac{\exp(\mathbf{u}_t \cdot \mathbf{v}_c)}{\sum_{k \in \mathcal{V}} \exp(\mathbf{u}_t \cdot \mathbf{v}_k)} \right)$$

- We need to compute the gradient of y with respect to $\mathbf{u}_t$ and $\mathbf{v}_k$, $\mathbf{u}_t \in \mathcal{V}$

Natural Language Processing (COSE461)

Word Embedding

Word2vec

Let's compute gradients for word2vec

$$y = -\log\left(\frac{\exp(\mathbf{u}_t \cdot \mathbf{v}_c)}{\sum_{k \in \mathcal{V}} \exp(\mathbf{u}_t \cdot \mathbf{v}_k)}\right)$$

$$\begin{aligned} y &= -\log(\exp(\mathbf{u}_t \cdot \mathbf{v}_c)) + \log(\sum_{k \in \mathcal{V}} \exp(\mathbf{u}_t \cdot \mathbf{v}_k)) \\ &= -\mathbf{u}_t \cdot \mathbf{v}_c + \log(\sum_{k \in \mathcal{V}} \exp(\mathbf{u}_t \cdot \mathbf{v}_k)) \end{aligned}$$

- Recall that

$$P(w_{t+j}|w_t) = \frac{\exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_{w_{t+j}})}{\sum_{k \in \mathcal{V}} \exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_k)}$$

$$\frac{\partial y}{\partial \mathbf{u}_t} = \frac{\partial(-\mathbf{u}_t \cdot \mathbf{v}_c)}{\partial \mathbf{u}_t} + \frac{\partial(\log \sum_{k \in \mathcal{V}} \exp(\mathbf{u}_t \cdot \mathbf{v}_k))}{\partial \mathbf{u}_t}$$

$$= -\mathbf{v}_c + \frac{\frac{\partial \sum_{k \in \mathcal{V}} \exp(\mathbf{u}_t \cdot \mathbf{v}_k)}{\partial \mathbf{u}_t}}{\sum_{k \in \mathcal{V}} \exp(\mathbf{u}_t \cdot \mathbf{v}_k)}$$

$$= -\mathbf{v}_c + \frac{\sum_{k \in \mathcal{V}} \exp(\mathbf{u}_t \cdot \mathbf{v}_k) \cdot \mathbf{v}_k}{\sum_{k \in \mathcal{V}} \exp(\mathbf{u}_t \cdot \mathbf{v}_k)}$$

$$= -\mathbf{v}_c + \sum_{k \in \mathcal{V}} \frac{\exp(\mathbf{u}_t \cdot \mathbf{v}_k)}{\sum_{k \in \mathcal{V}} \exp(\mathbf{u}_t \cdot \mathbf{v}_k)} \mathbf{v}_k$$

$$= -\mathbf{v}_c + \sum_{k \in \mathcal{V}} P(k|t) \mathbf{v}_k$$

Natural Language Processing (COSE461)

워드 임베딩

Word2vec

- What about context vectors?

$$\frac{\partial y}{\partial \mathbf{v}_k} = \begin{cases} (P(k|t) - 1) \mathbf{u}_t, & k = c \\ P(k|t) \mathbf{u}_t, & k \neq c \end{cases}$$

$$y = -\log\left(\frac{\exp(\mathbf{u}_t \cdot \mathbf{v}_c)}{\sum_{k \in \mathcal{V}} \exp(\mathbf{u}_t \cdot \mathbf{v}_k)}\right)$$

Natural Language Processing (COSE461)

Word Embedding

Word2vec

Overall algorithm

- 입력: 텍스트 코퍼스, 임베딩 크기 d , 어휘 V , 컨텍스트 크기 m
- 초기화: u_i, v_j 를 무작위로 초기화 $A_i \in V$
- 훈련 코퍼스를 순회하며 각 훈련 인스턴스 (t, c) 에 대해:

$$\text{update} \quad \mathbf{u}_t \leftarrow \mathbf{u}_t - \eta \frac{\partial y}{\partial \mathbf{u}_t} \quad \frac{\partial y}{\partial \mathbf{u}_t} = -\mathbf{v}_c + \sum_{k \in V} P(k|t) \mathbf{v}_k$$

$$\text{update} \quad \mathbf{v}_k \leftarrow \mathbf{v}_k - \eta \frac{\partial y}{\partial \mathbf{v}_k} \quad \frac{\partial y}{\partial \mathbf{v}_k} = \begin{cases} (P(k|t) - 1) \mathbf{u}_t, & k = c \\ P(k|t) \mathbf{u}_t, & k \neq c \end{cases}$$

Q: Can you think of any issues with this algorithm?

Natural Language Processing (COSE461)

Word Embedding

Word2vec

Skip-gram with negative sampling (SGNS)

- Problem: every time you get one pair of (t, c) , you need to update V_k with all the words in the vocabulary! This is very expensive computationally.

$$\frac{\partial y}{\partial \mathbf{u}_t} = -\mathbf{v}_c + \sum_{k \in V} P(k|t) \mathbf{v}_k \quad \frac{\partial y}{\partial \mathbf{v}_k} = \begin{cases} (P(k|t) - 1) \mathbf{u}_t & k = c \\ P(k|t) \mathbf{u}_t & k \neq c \end{cases}$$

- Negative sampling: instead of considering all the words in V , let's randomly sample K (5-20) negative examples.

$$\text{softmax: } y = -\log\left(\frac{\exp(\mathbf{u}_t \cdot \mathbf{v}_c)}{\sum_{k \in V} \exp(\mathbf{u}_t \cdot \mathbf{v}_k)}\right)$$

K

negative sampling: $y = -\log((\mathbf{u}_t \cdot \mathbf{v}_c)) - \sum_{j=1}^K \log((-\mathbf{u}_t \cdot \mathbf{v}_{j_i}))$

$i=1$

워드 임베딩

Word2vec

- Key idea: Convert the $|V|$ -way classification into a set of binary classification tasks.
- Every time we get a pair of words (t, c) , we don't predict C among all the words in the vocabulary.
- Instead, we predict (t, c) is a positive pair, and (t, c') is a negative pair for a small number of sampled c' . positive examples + negative examples -

w	c_{pos}
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

w	c_{neg}	w	c_{neg}
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if

- Similar to binary logistic regression, but we need to optimize u and V together.

$$P(y = 1 | t, c) = (\mathbf{u}_t \cdot \mathbf{v}_c) \quad P(y = 0 | t, c') = 1 - (\mathbf{u}_t \cdot \mathbf{v}_{c'}) = (-\mathbf{u}_t \cdot \mathbf{v}_{c'})$$

워드 임베딩

Word2vec

Understanding SGNS

K

$$y = -\log((u_t \cdot v_c)) - \sum_{i=1}^K \log((-u_t \cdot v_{j_i}))$$

i=1

- In skip-gram with negative sampling (SGNS), how many parameters need to be updated in for every (t,c) pair?

- (A) Kd
- (B) 2Kd
- (C) (K+1)d
- (D) (K+2)d

워드 임베딩

Word2vec

Continuous Bag of Words (CBOW)

$$L() = -\sum_{j=-m}^m P(w_t | \{w_{t+j}\}_{j \neq 0})$$

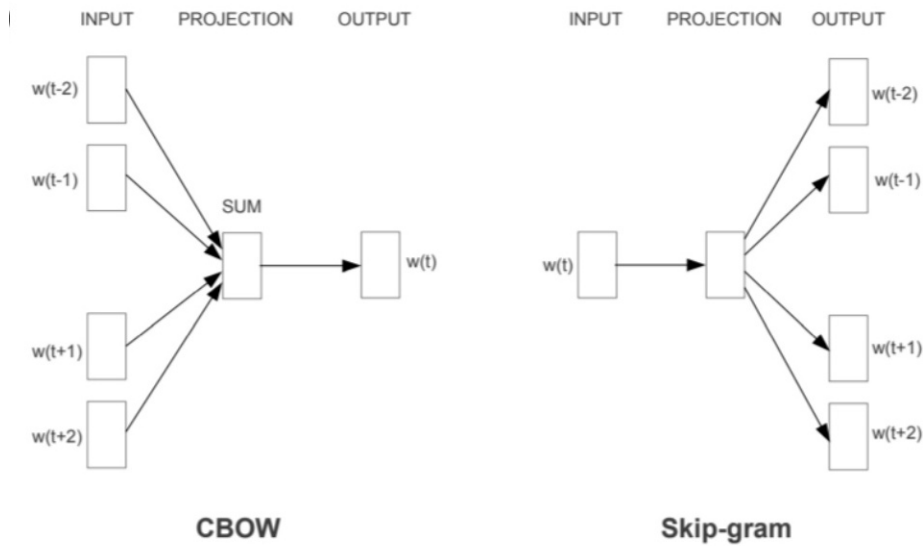
$$D_t = \sum_{j=-m}^m \sum_{j \neq 0} V_{t+j}$$

2m

$\exp(uw^t \cdot D^t)$

$P(w^t | \{w^{t+j}\})$

$\text{Ske} \exp(u_k \cdot v^t)$



Natural Language Processing (COSE461)

Word Embedding

Word2vec

GloVe: Global Vectors

- Take the global co-occurrence statistics: $X_{i,j}$
- Key idea: let's approximate $u_i \cdot V_j$ using their co-occurrence counts directly

$$J() = f(X_{i,j})(u_i \cdot V_j + b_i + b_j - \log X_{i,j})$$

$i, j \in V$

f : Weighting function, want to give more importance to more common pairs, but capped

at a certain point

- Advantages:

Training faster

Scalable too very large corpora

Natural Language Processing (COSE461)

Word Embedding

워드투벡

FastText: Subword Embeddings

- Similar to Skip-gram, but break words into n-grams with $n = 3$ to 6

3-grams: <wh, whe, her, ere, re>

4-grams: <whe, wher, here, ere>

5-grams: <wher, where, here>

6-grams: <where, where>

- Replace $u_i \cdot V_j$ by Egen-grams(w_i) $u_g \cdot V_j$

- 장점:

Uses subword information to handle OOV words and rare terms

Improves word similarity by capturing character-level features

unhappiness

= un + happy + ness

Natural Language Processing (COSE461)

Word Embedding

Word2vec

쉽게 사용할 수 있음

```
from gensim.models import KeyedVectors
# Load vectors directly from the file
model = KeyedVectors.load_word2vec_format('data/GoogleGoogleNews-vectors-negative300.bin', binary=True)
# Access vectors for specific words with a keyed lookup:
vector = model['easy']
```

```
In [17]: model.similarity('straightforward', 'easy')
```

```
Out[17]: 0.5717043285477517
```

```
In [18]: model.similarity('simple', 'impossible')
```

```
Out[18]: 0.29156160264633707
```

```
In [19]: model.most_similar('simple')
```

```
Out[19]: [('straightforward', 0.7460169196128845),
 ('Simple', 0.7108174562454224),
 ('uncomplicated', 0.6297484636306763),
 ('simplest', 0.6171397566795349),
 ('easy', 0.5990299582481384),
 ('fairly_straightforward', 0.5893306732177734),
 ('deceptively_simple', 0.5743066072463989),
 ('simpler', 0.5537199378013611),
 ('simplistic', 0.5516539216041565),
 ('disarmingly_simple', 0.5365327000617981)]
```

```
In [17]: model.similarity('straightforward', 'easy')
```

```
Out[17]: 0.5717043285477517
```

```
In [18]: model.similarity('simple', 'impossible')
```

```
Out[18]: 0.29156160264633707
```

```
In [19]: model.most_similar('simple')
```

```
Out[19]: [('straightforward', 0.7460169196128845),
 ('Simple', 0.7108174562454224),
 ('uncomplicated', 0.6297484636306763),
 ('simplest', 0.6171397566795349),
 ('easy', 0.5990299582481384),
 ('fairly_straightforward', 0.5893306732177734),
 ('deceptively_simple', 0.5743066072463989),
 ('simpler', 0.5537199378013611),
 ('simplistic', 0.5516539216041565),
 ('disarmingly_simple', 0.5365327000617981)]
```

Natural Language Processing (COSE461)

Evaluating word nbeddings

Word Embedding

Evaluating Word Embeddings

Extrinsic VS intrinsic evaluation

외재적 평가

Let's plug these word embedding into a real NLP system and see whether this improves performance

Could take a long time but still the most important evaluation metric

- Intrinsic evaluation

Evaluate on a specific/intermediate subtask

Fast to compute

Not clear if it really helps downstream tasks.

Natural Language Processing (COSE461)

Word Embedding

단어 임베딩 평가

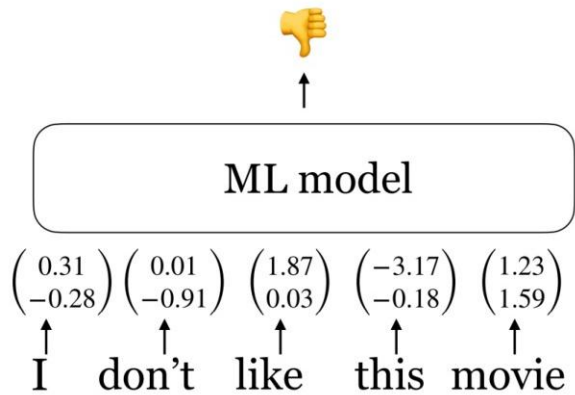
Extrinsic evaluation

- A straightforward solution: given an input sentence $X_1, X_2, \dots, X_n$

- Instead of using a bag-of-words model, we can compute $\text{vec}(x) = e(x_1) + e(x_2) + \dots + e(x_n)$

- And then train a logistic regression classifier on $\text{vec}(x)$ as we did before!

외재적 평가



There are much better ways to do this e.g., take word embeddings as input of neural networks

Natural Language Processing (COSE461)

Word Embedding

Evaluating Word Embeddings

Intrinsic evaluation: word similarity

- 워드 임베딩 유사도 측정 데이터셋: wordsim-353
- 353 pairs of words with human judgment

Word 1	Word 2	Human (mean)
tiger	cat	7.35
tiger	tiger	10
book	paper	7.46
computer	internet	7.58
plane	car	5.77
professor	doctor	6.62
stock	phone	1.62
stock	CD	1.31
stock	jaguar	0.92

Cosine similarity:

$$\cos(\mathbf{u}_i, \mathbf{u}_j) = \frac{\mathbf{u}_i \cdot \mathbf{u}_j}{\|\mathbf{u}_i\|_2 \times \|\mathbf{u}_j\|_2}.$$

Metric: Spearman rank correlation

Natural Language Processing (COSE461)

Word Embedding

단어 임베딩 평가

Model	Size	WS353	MC	RG	SCWS	RW
SVD	6B	35.3	35.1	42.5	38.3	25.6
SVD-S	6B	56.5	71.5	71.0	53.6	34.7
SVD-L	6B	65.7	<u>72.7</u>	75.1	56.5	37.0
CBOW [†]	6B	57.2	65.6	68.2	57.0	32.5
SG [†]	6B	62.8	65.2	69.7	<u>58.1</u>	37.2
GloVe	6B	<u>65.8</u>	<u>72.7</u>	<u>77.8</u>	53.9	<u>38.1</u>
SVD-L	42B	74.0	76.4	74.1	58.3	39.9
GloVe	42B	<u>75.9</u>	<u>83.6</u>	<u>82.9</u>	<u>59.6</u>	<u>47.8</u>
CBOW*	100B	68.4	79.6	75.4	59.4	45.5

SG: Skip-gram

Natural Language Processing (COSE461)

Word Embedding

Evaluating Word Embeddings

내재적 평가: 단어 유추

* b: b*

- Word analogy test: a: a =

$b * \operatorname{argmax}_{w \in V} \cos(e(w), e(a^*) - e(a) + e(b))$

=

- Semantic:

시카고 : 일리노이 = 필라델피아 : ?

- Syntactic

bad : worst = cool : ?

Model	Dim.	Size	Sem.	Syn.	Tot.
ivLBL	100	1.5B	55.9	50.1	53.2
HPCA	100	1.6B	4.2	16.4	10.8
GloVe	100	1.6B	<u>67.5</u>	<u>54.3</u>	<u>60.3</u>
SG	300	1B	61	61	61
CBOW	300	1.6B	16.1	52.6	36.1
vLBL	300	1.5B	54.2	<u>64.8</u>	60.0
ivLBL	300	1.5B	65.2	63.0	64.0
GloVe	300	1.6B	<u>80.8</u>	61.5	<u>70.3</u>
SVD	300	6B	6.3	8.1	7.3
SVD-S	300	6B	36.7	46.6	42.1
SVD-L	300	6B	56.6	63.0	60.1
CBOW [†]	300	6B	63.6	<u>67.4</u>	65.7
SG [†]	300	6B	73.0	66.0	69.1
GloVe	300	6B	<u>77.4</u>	67.0	<u>71.7</u>
CBOW	1000	6B	57.3	68.9	63.7
SG	1000	6B	66.1	65.1	65.6
SVD-L	300	42B	38.4	58.2	49.2
GloVe	300	42B	<u>81.9</u>	<u>69.3</u>	<u>75.0</u>

Metric: accuracy

- More examples at <http://download.tensorflow.org/data/questions>

Natural Language Processing (COSE461)

E.O.D